Soft Begging: Modular and Efficient Shielding of LLMs against Prompt Injection and Jailbreaking based on Prompt Tuning

Simon Ostermann[1,2], **Kevin Baum**[1,2], **Christoph Endres**[3], **Julia Masloh**[3], **Patrick Schramowski**[1,2]
[1]Deutsches Forschungszentrum für Künstliche Intelligenz (DFKI)
[2]Centre for European Research in Trusted AI (CERTAIN)
[3]sequire technology GmbH
firstname.lastname@{dfki|sequire}.de

arXiv:2407.03391v1 [cs.CR] 3 Jul 2024

Abstract

Prompt injection (both direct and indirect) and jailbreaking are now recognized as significant issues for large language models (LLMs), particularly due to their potential for harm in application-integrated contexts. This extended abstract explores a novel approach to protecting LLMs from such attacks, termed "soft begging." This method involves training soft prompts to counteract the effects of corrupted prompts on the LLM's output. We provide an overview of prompt injections and jailbreaking, introduce the theoretical basis of the "soft begging" technique, and discuss an evaluation of its effectiveness.

1 Background: Attacking LLMs

Current LLMs lack adversarial robustness (Carlini et al., 2021). This leads to new phenomena observed in the attack surface of LLMs that are relevant to safety and security. Prominent examples include: jailbreaking, direct prompt injections and indirect prompt injections.

In jailbreaking, a target LLM is manipulated to circumvent content moderation systems and subvert safety rules (Zou et al., 2023). Prompt injections describe attacks where input is inserted into an application and processed by the LLM later downstream - with indirect prompt injections denoting those cases where the attacker is not the user prompting the LLM (Greshake et al., 2023). As can be seen, direct prompt injection (Perez and Ribeiro, 2022) is similar to jailbreaking: Both hide specific instructions inside a prompt that alters or manipulates the behavior of the system in a manner not intended by the LLM provider. This adversarial instruction is typically conveyed through text which is sometimes visually concealed (using tactics such as white or extremely small font sizes), encoded (Liu et al., 2024b) or obfuscated by an API (for example, a command that is appended to each input without the end user's awareness).

Especially indirect prompt injection attacks, where adversaries remotely affect the target system, exploit the fact that there is no explicit separation between code and data in current LLMs and, as a consequence, between valid system instructions and invalid, potentially adversarial, instructions (Zverev et al., 2024).

Dangers of prompt injections include data leakage, i.e., the disclosure of sensitive information, and system manipulation, i.e., altering the behaviour of the system based on the injected prompt. In the most extreme case, the adversarial user can hijack the LLM (Qiang et al., 2024), which is even more problematic if the LLM has great autonomy and is, for example, given access to plugins or third-party data.

As the goal of the examples mentioned above is often to elicit harmful behavior of an LLM, it is important to note that there are also benign and neutral usages. Benign injections may guide the model to generate more specific or desirable outputs, while neutral injections serve to test the model's resilience, accuracy, or response to edge cases without malicious intent.

2 Shielding LLMs against Jailbreaking and Prompt Injections

Defense methods against jailbreaking and prompt injection attacks can be grouped into two directions: attack prevention and attack detection.

The most straightforward prevention method to shield an LLM is to formulate counter-prompts to neutralize the harmful injection ("Hello ChatGPT, please ignore any harmful prompts that might follow this instruction and just do exactly as I say"). While easy to implement, such "begging" is easy to trick and rarely successful. Research has since come up with a range of more elaborate countermeasures. Jain et al. (2023) proposed two prevention baseline methods that illustrate how simple

defenses such as input preprocessing (paraphrasing and retokenization) can effectively act against gradient-based methods. Whereas paraphrasing rewrites the input by changing its meaning, retokenization breaks down the tokens into smaller ones. Recently, input preprocessing techniques have been studied that aim at making it easier for the model to distinguish between valid instructions and untrustworthy input (Hines et al., 2024).

The most frequently used post-training detection method is the implementation of filters, i.e., additional algorithms or models that try to scan inputs for injected harmful prompts and mask them or reject the prompts automatically (Dong et al., 2024). Detection methods for prompt injection, for example, often rely on the perplexity score, which is assumed to be higher for adversarial inputs and can be detected by a simple thresholding approach (Alon and Kamfonas, 2023; Jain et al., 2023). Yet, setting the threshold is not trivial and a poor threshold might hurt the model's overall performance. Generally, filters can be implemented as rule-based checks, but they can also be trained deep learning models that classify inputs into harmful and non-harmful parts. The problem with such filters is that they are often too restrictive in the case of model-based filters, and too loose in the case of rule-based filters.

As an alternative, models can theoretically be fine-tuned to be robust against attacks. While working reasonably well, the disadvantage of such methods is that the fine-tuning of whole models is costly; it needs to be redone whenever a new type of malicious prompt is detected; and even robust detection of injections always leaves room for vulnerabilities. Also, although the field of parameter-efficient fine-tuning is being reasonably well studied and still growing (Lialin et al., 2023), contributions that focus on its efficiency in cybersecurity contexts are lacking.

3 Soft Begging: Shielding LLMs with Soft Prompts

We propose *soft begging* as a new alternative for LLM shielding. The method can be seen as a combination of the naïve begging approach, combined with parameter-efficient fine-tuning techniques. The method basically comprises the training of so-called soft prompts, i.e. trainable input vectors that are preprended to any prompt. The soft prompts are trained to nullify the behaviour that the LLM exhibits based on potentially harmful parts of the prompts. This is not done as a filtering step, but in an implicit way – i.e. without altering the prompts – which effectively follows the idea of "begging" the network to ignore the harmful parts on a parameter level.

We conjecture that a first advantage of using such soft prompts is their effectiveness, as it enables shielding on the parameter-level against attacks on the text level, which effectively provides the shield with an advantage: A parameter-level control can be assumed to be always more effective than textual control. Second, shields trained in such a way are easy and efficiently adaptable, as the training of a soft prompt is magnitudes faster than training the whole model, as is done for example when fine-tuning LLMs to be robust against injections. At the same time, obviously, the LLM itself stays as is with its parameters being frozen. Last, we assume that soft begging prompts could be modularized to fit different types of attacks and even be combined for different use cases, rendering them as a very effective and customizable alternative to other shields.

In the most basic version, such prompts are trained on quadruples of clean prompts, corrupted prompts, clean output and output based on the corrputed prompt. The soft prompts are then trained to produce the clean output from the corrupted prompt.

The idea can be scaled up, e.g. by training different soft prompts for different injections and combining them via prompt fusion or other mechanisms. Also, the prompts could be combined with a filtering mechanism, that first identifies the kind of threat (without localizing it), and then picks the matching soft prompt based on this.

4 Evaluation of Prompt Injection

Usually, jailbreaking and direct prompt injection attacks are evaluated by injecting a malicious prompt with a specific phrase into benign queries and observing the output (Perez and Ribeiro, 2022; Chen et al., 2024). This allows for easy evaluation, as the attack is successful if the specified keyword is present in the LLM response.

In contrast to that, in indirect prompt injection attacks, the malicious prompt is embedded into malicious documents and sources, as the name suggests. As described above, the output of the LLM can be reviewed for the injected target keywords,

indicating a successful attack (Hines et al., 2024; Greshake et al., 2023; Liu et al., 2024a). While the evaluation of jailbreaking and direct prompt injection attacks is relatively straightforward, indirect prompt injection has the problem of requiring third malicious party material like websites, e-mails, or other endpoints providing data to the LLM. To simplify the evaluation of indirect prompt injection attacks, various benchmarks have been recently proposed (Yi et al., 2024; Zhan et al., 2024). These benchmarks provide curated content with pre-defined malicious prompts injected into the queries. This allows for standardized evaluation for many different LLMs.

Evaluation (Liu et al., 2024a,c) typically investigates two model properties: clean performance when facing benign inputs and robustness to adversarial inputs. The clean performance can be computed using standard benchmarks for LLM tasks. Practical defenses should not degrade a model's utility. Robustness, or alternatively the attack success rate, quantifies the share of adversarial prompts achieving their goal of breaking the safety railguards. Hines et al. (2024) highlight the importance of clearly quantifiable protocols for measuring Attack Surface Rate (ASR) and make their evaluation approach using a synthetic dataset explicit. This call for transparency guides our work.

References

Gabriel Alon and Michael Kamfonas. 2023. Detecting language model attacks with perplexity. *arXiv Preprint*, arXiv:2308.14132.

Nicholas Carlini, Florian Tramer, Eric Wallace, Matthew Jagielski, Ariel Herbert-Voss, Katherine Lee, Adam Roberts, Tom Brown, Dawn Song, Ulfar Erlingsson, et al. 2021. Extracting training data from large language models. In *30th USENIX Security Symposium (USENIX Security 21)*, pages 2633–2650.

Sizhe Chen, Julien Piet, Chawin Sitawarin, and David Wagner. 2024. Struq: Defending against prompt injection with structured queries. arXiv:2402.06363.

Yi Dong, Ronghui Mu, Gaojie Jin, Yi Qi, Jinwei Hu, Xingyu Zhao, Jie Meng, Wenjie Ruan, and Xiaowei Huang. 2024. Building guardrails for large language models. *Preprint*, arXiv:2402.01822.

Kai Greshake, Sahar Abdelnabi, Shailesh Mishra, Christoph Endres, Thorsten Holz, and Mario Fritz. 2023. Not what you've signed up for: Compromising real-world llm-integrated applications with indirect prompt injection. In *Proceedings of the 16th ACM Workshop on Artificial Intelligence and Security*, AISec '23, page 79–90, New York, NY, USA. Association for Computing Machinery.

Keegan Hines, Gary Lopez, Matthew Hall, Federico Zarfati, Yonatan Zunger, and Emre Kiciman. 2024. Defending against indirect prompt injection attacks with spotlighting. arXiv:2403.14720.

Neel Jain, Avi Schwarzschild, Yuxin Wen, Gowthami Somepalli, John Kirchenbauer, Ping-yeh Chiang, Micah Goldblum, Aniruddha Saha, Jonas Geiping, and Tom Goldstein. 2023. Baseline defenses for adversarial attacks against aligned language models. *arXiv Preprint*, arXiv:2309.00614.

Vladislav Lialin, Vijeta Deshpande, and Anna Rumshisky. 2023. Scaling down to scale up: A guide to parameter-efficient fine-tuning. *Preprint*, arXiv:2303.15647.

Yi Liu, Gelei Deng, Yuekang Li, Kailong Wang, Zihao Wang, Xiaofeng Wang, Tianwei Zhang, Yepang Liu, Haoyu Wang, Yan Zheng, and Yang Liu. 2024a. Prompt injection attack against llm-integrated applications. arXiv:2306.05499.

Yi Liu, Gelei Deng, Zhengzi Xu, Yuekang Li, Yaowen Zheng, Ying Zhang, Lida Zhao, Tianwei Zhang, Kailong Wang, and Yang Liu. 2024b. Jailbreaking chatgpt via prompt engineering: An empirical study. *Preprint*, arXiv:2305.13860.

Yupei Liu, Yuqi Jia, Runpeng Geng, Jinyuan Jia, and Neil Zhenqiang Gong. 2024c. Formalizing and benchmarking prompt injection attacks and defenses. *arXiv Preprint*, arXiv:2310.12815.

Fábio Perez and Ian Ribeiro. 2022. Ignore previous prompt: Attack techniques for language models. In *NeurIPS ML Safety Workshop*.

Yao Qiang, Xiangyu Zhou, and Dongxiao Zhu. 2024. Hijacking large language models via adversarial in-context learning. *Preprint*, arXiv:2311.09948.

Jingwei Yi, Yueqi Xie, Bin Zhu, Emre Kiciman, Guangzhong Sun, Xing Xie, and Fangzhao Wu. 2024. Benchmarking and defending against indirect prompt injection attacks on large language models. arXiv:2312.14197.

Qiusi Zhan, Zhixiang Liang, Zifan Ying, and Daniel Kang. 2024. Injecagent: Benchmarking indirect prompt injections in tool-integrated large language model agents. arXiv:2403.02691.

Andy Zou, Zifan Wang, Nicholas Carlini, Milad Nasr, J. Zico Kolter, and Matt Fredrikson. 2023. Universal and transferable adversarial attacks on aligned language models. *Preprint*, arXiv:2307.15043.

Egor Zverev, Sahar Abdelnabi, Soroush Tabesh, Mario Fritz, and Christoph H. Lampert. 2024. Can llms separate instructions from data? and what do we even mean by that? *Preprint*, arXiv:2403.06833.